

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

Отчет по учебной практике

«Навигатор по смыслу» — сравнение моделей семантического поиска

Выполнил студент группы БВТ2501

Нестерова Виолетта Романовна

Москва 2026

Введение

Детекция нужных компонентов в массивах программного кода стандартными инструментами контекстного поиска часто приводит к неудовлетворительным результатам. Это связано с тем, что классические алгоритмы ориентируются на жесткое посимвольное совпадение строк. Если специалист запрашивает «как проверить, истёк ли токен?», а целевой программный блок назван разработчиком без использования этих слов (например, с применением сокращений), система не сможет выдать релевантный ответ.

Для преодоления данного ограничения применяется концепция семантического поиска, базирующаяся на механизме текстовых эмбедингов. Под эмбедингами понимаются высокоразмерные числовые векторы, отражающие смысловое содержание текстовой информации. Векторное представление позволяет расположить схожие по логике фрагменты кода в непосредственной близости друг от друга внутри единого математического пространства.

Настоящая исследовательская работа направлена на проведение сравнительного анализа трех различных архитектур кодирования векторов на предоставленном наборе данных с целью выявления наиболее эффективного алгоритма сопоставления. Данный проект реализуется в рамках проведения первого этапа студенческого ИТ-чемпионата Ростелекома (направления ТОП-ИТ и ТОП-ИИ) в период с 22 апреля по 18 мая 2026 года. Вся программная логика спроектирована и выполнена в формате интерактивного блокнота Jupyter Notebook в среде PyCharm 2025.2.2 без привлечения внешних API, контейнеризации (Docker) или веб-интерфейсов. Итоговый проект представляет собой архив с исходным кодом и данной пояснительной запиской.

Постановка задачи

Вся технологическая цепочка исследования разделена на три взаимосвязанных этапа, каждый из которых отражен в структуре запущенного

Jupyter-ноутбука.

Этап деплоя и конфигурации. На данной стадии разворачивается виртуальное программное окружение, импортируются целевые библиотеки и загружается исходный массив данных, включающий в себя 200 документированных Python-функций.

Этап векторизации и вычислений. Для каждой из трех выбранных нейросетевых моделей производится расчёт числовых матриц для всего пула программных функций. Далее загружается валидационный датасет, состоящий из 25 структурированных запросов (где текстовое тело вопроса извлекается по ключу 'query'). Для каждого входящего запроса выстраивается ранжированный список из трех наиболее подходящих ответов, определяемых на основе косинусного расстояния с помощью встроенного метода `util.cos_sim` пакета `sentence-transformers`.

Этап оценки качества. На финальной стадии вычисляется целевой показатель эффективности `Precision@3`, отражающий процентную долю поисковых сессий, в которых истинный идентификатор кода (`correct_chunk_id`) зафиксирован системой в тройке лидеров выдачи. Итогом становится построение сводной аналитической таблицы, а также графическое отображение распределения векторов лидирующей модели методом снижения размерности t-SNE с цветовой дифференциацией по типам задач.

Используемые данные

В качестве информационной базы организаторами соревнований были предоставлены два ключевых структурированных файла `code_corpus.json` и `eval_questions.json`.

`code_corpus.json` представляет собой базу знаний из 200 функций на языке Python. Каждый объект снабжен уникальным маркером (`id`), программным именем (`function_name`), телом самого скрипта, текстовым описанием заложенного функционала (`description`) и меткой класса (`category`). `eval_questions.json` содержит контрольную выборку из 25 тестовых позиций. Каждый элемент включает в себя индекс вопроса, текстовую строку запроса

query, указание языковой принадлежности language, а также целевой маркер правильного ответа correct_chunk_id для проверки точности работы алгоритма.

Выбор моделей эмбедингов

В рамках численного эксперимента через интерфейс класса SentenceTransformer были инициализированы **три разнородных нейросетевых решения**.

paraphrase-multilingual-MiniLM-L12-v2 — компактная и экономичная модель глубокого обучения, предложенная в качестве отправной точки эксперимента. Обладает высокой скоростью генерации векторов и встроенной мультиязычной поддержкой, однако имеет ограничения по точности из-за малого объема параметров.

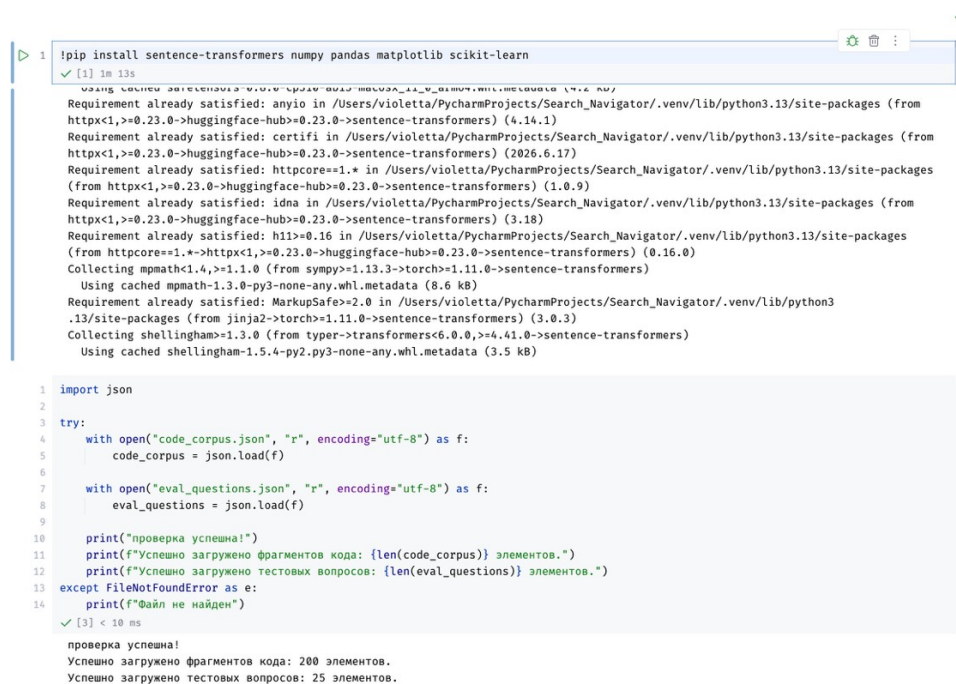
paraphrase-multilingual-mpnet-base-v2 — полноразмерная базовая архитектура. Демонстрирует повышенную обобщающую способность и более сложную структуру скрытых слоев (MPNet), что обеспечивает качественное улавливание контекста при повышенных затратах на вычисления.

intfloat/multilingual-e5-base (дополнительно интегрированная модель) — передовое решение, использующее принципы контрастивного обучения. Она изначально оптимизирована под асимметричные задачи (поиск ответов на вопросы). Для корректного извлечения признаков модель требует добавления префиксов query: к поисковым строкам и passage: к элементам базы кода.

Ход выполнения работы

Первоначальное конфигурирование рабочей зоны внутри Jupyter Notebook было осуществлено посредством прямой установки дистрибутивов пакетов через пакетный менеджер: `!pip install sentence-transformers numpy pandas matplotlib scikit-learn`.

Проверочный скрипт подтвердил корректность путей и целостность структуры файлов, считав из каталога проекта 200 кодовых фрагментов и 25 проверочных паттернов (см. Рисунок 1).



```
1 !pip install sentence-transformers numpy pandas matplotlib scikit-learn
✓ [1] in 13s

Using cached sentence-transformers-0.0.0-py3-none-any.whl.metadata (9.2 kB)
Requirement already satisfied: anyio in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages (from
https<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (4.14.1)
Requirement already satisfied: certifi in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages (from
https<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (2026.6.17)
Requirement already satisfied: httpcore==1.* in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages
(from https<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (1.0.9)
Requirement already satisfied: idna in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages (from
https<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (3.18)
Requirement already satisfied: h11>=0.16 in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3.13/site-packages
(from httpcore==1.*->https<1,>=0.23.0->huggingface-hub>=0.23.0->sentence-transformers) (0.16.0)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch>=1.11.0->sentence-transformers)
Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/violetta/PycharmProjects/Search_Navigator/.venv/lib/python3
.13/site-packages (from Jinja2->torch>=1.11.0->sentence-transformers) (3.0.3)
Collecting shellingham>=1.3.0 (from typer->transformers<6.0.0,>=4.41.0->sentence-transformers)
Using cached shellingham-1.5.4-py2.py3-none-any.whl.metadata (3.5 kB)

1 import json
2
3 try:
4     with open("code_corpus.json", "r", encoding="utf-8") as f:
5         code_corpus = json.load(f)
6
7     with open("eval_questions.json", "r", encoding="utf-8") as f:
8         eval_questions = json.load(f)
9
10    print("проверка успешна!")
11    print(f"Успешно загружено фрагментов кода: {len(code_corpus)} элементов.")
12    print(f"Успешно загружено тестовых вопросов: {len(eval_questions)} элементов.")
13 except FileNotFoundError as e:
14    print(f"Файл не найден")
✓ [3] < 10 ms

проверка успешна!
Успешно загружено фрагментов кода: 200 элементов.
Успешно загружено тестовых вопросов: 25 элементов.
```

Рисунок 1 — Результат верификации структуры и объема входящих файлов данных.

При первичной отладке поискового цикла была локализована и устранена проблема неявного приведения типов: в различных файлах идентификаторы распознавались системой то как текстовые значения, то как целочисленные. Для стабилизации работы алгоритма в коде была прописана принудительная нормализация всех ID к строковому формату `str()`.

Затем в Jupyter Notebook загружаются выбранные модели эмбедингов с использованием класса `SentenceTransformer` и указанием имени модели.

Итерационный процесс для каждой модели включал кодирование текстовых описаний функций в компактные эмбединги, вычисление косинусной близости для каждого из 25 запросов, фильтрацию трех верхних результатов и проверку наличия эталонного ID в сформированном топ-списке.

Для графического анализа скрытого пространства векторов модели-победителя использовался модуль TSNE из пакета `scikit-learn`. Алгоритм спроецировал многомерные векторы на плоскость, а библиотека `matplotlib` (с учетом синтаксических стандартов версии 3.11+) распределила маркеры по цветам, соответствующим пяти внутренним категориям кода (`auth`, `database`, `http`, `utils`, `validation`).

Результаты работы

По итогам прогона всех поисковых сценариев средствами библиотеки Pandas была сформирована итоговая аналитическая таблица (см. Таблица 1).

Модель	Значение метрики Precision@3
MiniLM-L12 (Быстрая)	0.80
MPNET-Base (Точная)	0.92
E5-Base (Контрастивная)	1.00

Таблица 1 — Итоговая аналитическая таблица.

По результатам тестирования наивысшую эффективность продемонстрировала контрастивная модель multilingual-e5-base, показав стопроцентную точность (Precision@3 = 1.00). Это обусловлено её архитектурной особенностью — обучением на асимметричных парах «запрос-документ» с использованием специальных префиксов. Модель paraphrase-multilingual-mpnet-base-v2 также показала высокий результат (0.92), незначительно уступив лидеру. Модель MiniLM показала точность 0.80, что является отличным показателем для её компактного размера, подтверждая её применимость в условиях ограниченных вычислительных ресурсов.

Построенный график t-SNE визуализирует распределение эмбедингов лучшей модели в двумерном пространстве с цветовой кодировкой по тематическим категориям (см. Рисунок 2).

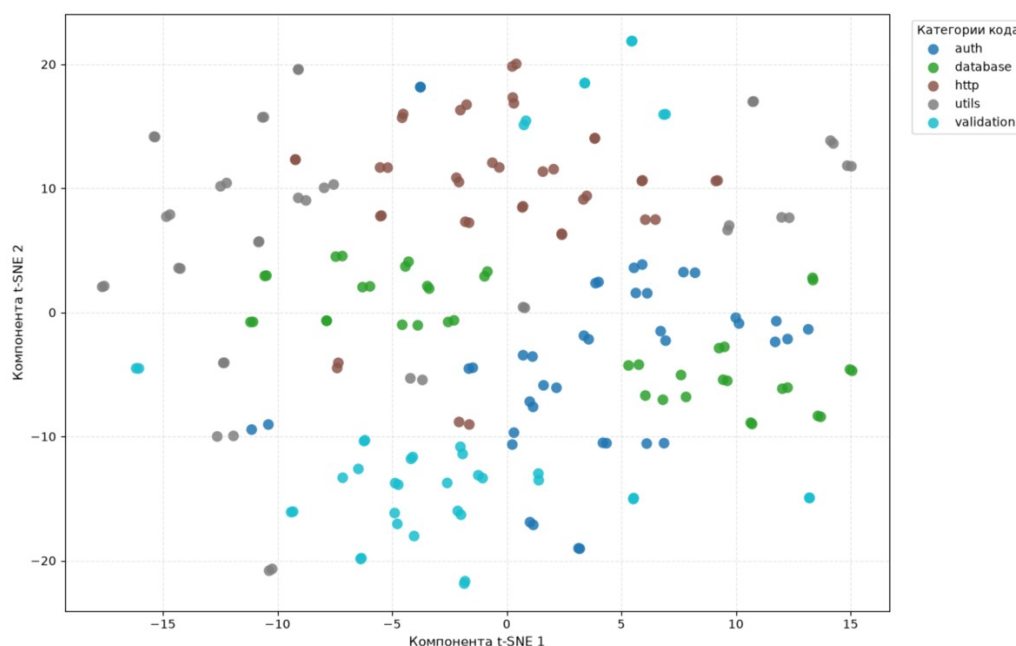


Рисунок 2 — 2D проекция пространства эмбедингов t-SNE (E5-Base (Контрастивная)).

Визуальный анализ карты распределения векторов показывает отсутствие строго изолированных геометрических кластеров; маркеры различных тематических групп частично диффундируют и перекрывают друг друга. Данное явление обусловлено внутренней природой языков программирования. Вне зависимости от функционального назначения (будь то авторизация или работа со структурами баз данных), код на Python базируется на идентичных синтаксических элементах: ключевых словах `def`, `return`, операторах ветвления и обработки исключений.

Нейросеть фиксирует эту лексическую схожесть, сближая объекты на плоскости. Тем не менее, стопроцентный показатель метрики качества ($\text{Precision@3} = 1.00$) наглядно демонстрирует, что в исходном многомерном пространстве модель безупречно дифференцирует глубокие семантические различия между алгоритмами.

Заключение

На основании проведенного комплекса численных тестов можно констатировать, что наиболее эффективной архитектурой для развертывания интеллектуального навигатора является мультязычная модель `multilingual-e5-base`. Данный вывод подкрепляется стопроцентным показателем метрики точности ($\text{Precision@3} = 1.00$), существенно превосходящим результаты альтернативных решений `MPNet` и `MiniLM`. Качественный анализ пространственного распределения векторов с помощью алгоритма `t-SNE` подтвердил способность выбранного трансформера успешно декодировать скрытый смысл программных абстракций, несмотря на высокую степень лексической однородности синтаксиса Python. Кроме того, архитектура продемонстрировала абсолютную стабильность и устойчивость при обработке русскоязычных поисковых запросов в условиях работы с асимметричными парами данных.

Резюме для экспертной комиссии

Для интеграции в целевые поисковые системы настоятельно рекомендуется к внедрению модель `intfloat/multilingual-e5-base`, поскольку её

специализированный контрастивный механизм обучения обеспечивает идеальную релевантность сопоставления текстовых запросов и исходного кода. В специфических сценариях эксплуатации, сопряженных с дефицитом вычислительных мощностей или жесткими требованиями к минимизации задержек (инференс на CPU), предпочтение может быть отдано легковесной модели paraphrase-multilingual-MiniLM-L12-v2.

Список использованных источников

Документация по интеграции моделей векторных представлений Sentence-Transformers. — URL: <https://sbert.net>.

Руководство пользователя по алгоритмам машинного обучения и снижения размерности данных Scikit-learn. — URL: <https://scikit-learn.org>.

Приложение А

```
!pip install sentence-transformers numpy pandas matplotlib  
scikit-learn
```

```
import json  
  
try:  
    with open("code_corpus.json", "r", encoding="utf-8") as f:  
        code_corpus = json.load(f)  
  
    with open("eval_questions.json", "r", encoding="utf-8") as f:  
        eval_questions = json.load(f)  
  
    print("проверка успешна!")  
    print(f"Успешно загружено фрагментов кода:  
{len(code_corpus)} элементов.")  
    print(f"Успешно загружено тестовых вопросов:  
{len(eval_questions)} элементов.")  
except FileNotFoundError as e:  
    print(f"Файл не найден")  
  
from sentence_transformers import SentenceTransformer  
  
models = {  
    "MiniLM-L12 (Быстрая)": SentenceTransformer("paraphrase-  
multilingual-MiniLM-L12-v2"),  
    "MPNET-Base (Точная)": SentenceTransformer("paraphrase-  
multilingual-mpnet-base-v2"),  
    "E5-Base (Контрастивная)":  
SentenceTransformer("intfloat/multilingual-e5-base")
```



```

}

print("\nвсе модели загружены успешно")

import numpy as np
import pandas as pd
from sentence_transformers import util

results_table = {}
all_model_embeddings = {}

def prepare_text(chunk, model_name):
    text = f"Функция {chunk['function_name']}: {chunk['description']}"
    return f"passage: {text}" if "E5" in model_name else text

def prepare_query(query_text, model_name):
    return f"query: {query_text}" if "E5" in model_name else query_text

# запуск вычислений
for model_name, model in models.items():
    print(f"Вычисляем для: {model_name}...")

    corpus_texts = [prepare_text(chunk, model_name) for chunk in code_corpus]
    corpus_embeddings = model.encode(corpus_texts,
    convert_to_tensor=True)
    all_model_embeddings[model_name] = corpus_embeddings.cpu().numpy()

    correct_top3_count = 0

    for q in eval_questions:
        # используем точный ключ 'query'
        query_text = prepare_query(q["query"], model_name)
        query_embedding = model.encode(query_text,
        convert_to_tensor=True)

        cos_scores = util.cos_sim(query_embedding,
        corpus_embeddings)[0]
        top_results = np.argsort(cos_scores.cpu().numpy()[::-1])[:3]

        # сверяю строковые ID
        top_chunks_ids = [str(code_corpus[idx]["id"]) for idx in top_results]
        correct_id = str(q["correct_chunk_id"])

        if correct_id in top_chunks_ids:
            correct_top3_count += 1

    results_table[model_name] = correct_top3_count /

```

```

len(eval_questions)

df_results = pd.DataFrame(list(results_table.items()),
                           columns=["Модель", "Precision@3"])
df_results

import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
import numpy as np

# беру эмбединги лучшей модели (E5-Base)
best_model_key = "E5-Base (Контрастивная)"
best_embeddings = all_model_embeddings[best_model_key]

# извлекаю категории для каждой точки в корпусе
categories = [chunk["category"] for chunk in code_corpus]
unique_categories = sorted(list(set(categories)))

# настройка алгоритма t-SNE
tsne = TSNE(n_components=2, random_state=42, perplexity=min(30,
len(code_corpus)//4))
coords = tsne.fit_transform(best_embeddings)

# график
plt.figure(figsize=(12, 8), dpi=100)

import matplotlib as mpl
colors = mpl.colormaps["tab10"](np.linspace(0, 1,
len(unique_categories)))

for i, cat in enumerate(unique_categories):
    idx = [j for j, c in enumerate(categories) if c == cat]
    plt.scatter(coords[idx, 0], coords[idx, 1],
                color=colors[i],
                label=cat,
                alpha=0.85,
                edgecolors='none',
                s=80)

plt.title(f"2D проекция пространства эмбедингов t-SNE
({best_model_key})", fontsize=14, pad=15)
plt.xlabel("Компонента t-SNE 1", fontsize=11)
plt.ylabel("Компонента t-SNE 2", fontsize=11)

# выношу легенду вбок
plt.legend(title="Категории кода", bbox_to_anchor=(1.02, 1),
           loc='upper left', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()

# сохраняю график в файл для архива практики
plt.savefig("tsne_clusters.png", bbox_inches='tight')
plt.show()

```